

Mata Kuliah Basis Data

Desain Basis Data dan Implementasinya Dalam *Small Business Inventory and Sales System*



Kelompok 13:

Muhammad Dafa 'Izzul Iman A. (24/535342/PA/22711)

Ivan Zuhri Ramadhani Syahrial (24/540342/PA/22939)

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA YOGYAKARTA**

2025

BAB 1: PENDAHULUAN

1.1 Latar Belakang Masalah

Dalam operasional Usaha Kecil dan Menengah (UKM), pengelolaan inventaris merupakan salah satu aspek paling krusial. Saat ini, banyak pelaku usaha yang masih mengandalkan pencatatan manual menggunakan buku besar atau *spreadsheet* sederhana (Excel). Metode ini memiliki berbagai kelemahan fatal, antara lain:

1. **Human Error:** Kesalahan pencatatan jumlah stok saat barang masuk atau keluar.
2. **Ketidakakuratan Data Real-time:** Sulit mengetahui sisa stok saat itu juga, menyebabkan kejadian kehabisan stok (*stockout*) saat pelanggan ingin membeli.
3. **Keamanan Data:** Tidak adanya pembatasan akses, sehingga siapa saja bisa mengubah data penjualan.
4. **Kesulitan Pelaporan:** Pemilik bisnis sulit merekapitulasi keuntungan dan tren penjualan secara cepat.

Berdasarkan permasalahan tersebut, dikembangkanlah aplikasi "**SmallBiz Inventory Management System**". Sistem ini dirancang untuk mendigitalisasi seluruh proses pencatatan stok, transaksi penjualan (*Point of Sales*), dan manajemen pemasok (*Supplier*) dalam satu platform berbasis web yang terintegrasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah yang diselesaikan dalam proyek ini adalah:

1. Bagaimana merancang basis data yang mampu menangani relasi antara produk, pemasok, pelanggan, dan transaksi secara efisien?
2. Bagaimana membangun logika *backend* untuk memastikan stok berkurang secara otomatis saat transaksi terjadi?
3. Bagaimana menyajikan visualisasi status stok (Aman/Menipis/Habis) yang akurat kepada pengguna?

1.3 Tujuan Pembahasan

1. **Membangun Sistem Terintegrasi:** Menghubungkan modul Inventory, Kasir, dan Manajemen User.
2. **Implementasi Keamanan:** Menerapkan otentikasi berbasis JSON Web Token (JWT) dan *Role-Based Access Control* (Admin, Manager, Cashier).
3. **Otomasi Inventaris:** Menerapkan pemicu otomatis (*trigger logic*) pada level aplikasi untuk memutakhirkan kuantitas stok.

1.4 Batasan Masalah

1. Sistem berjalan di lingkungan *localhost* menggunakan Node.js.
2. Pembayaran yang didukung saat ini adalah tunai (*Cash*).
3. Sistem mencakup manajemen stok barang fisik, bukan jasa.

BAB 2: ANALISIS DAN PERANCANGAN SISTEM

2.1 Analisis Kebutuhan Pengguna (User Requirement)

Sistem memiliki tiga aktor utama dengan hak akses berbeda:

1. **Admin:**
 - Memiliki akses penuh (*Super User*).
 - Dapat membuat, mengedit, dan menghapus akun pengguna (Manager/Cashier).
 - Dapat mengaktifkan/menonaktifkan status pengguna.
2. **Manager (Gudang/Inventaris):**
 - Mengelola Data Master Produk (Tambah, Edit, Hapus).
 - Mengelola Data Supplier dan Customer.
 - Melakukan *Restock* (Pembelian barang masuk).
 - Melihat laporan stok menipis (*Low Stock*).
3. **Cashier (Kasir):**
 - Melakukan transaksi penjualan sehari-hari.
 - Mencetak atau melihat detail struk transaksi.
 - Tidak memiliki akses untuk mengubah data stok secara langsung (harus melalui transaksi).

2.2 Perancangan Basis Data (Database Design)

Untuk menjamin integritas data, perancangan database menggunakan prinsip Normalisasi hingga Tahap Ketiga (3NF). Berikut adalah entitas utama dalam sistem:

2.2.1 Entity Relationship Diagram (ERD) Deskriptif

- **User** mengelola **Transaction** dan **Purchase**.
- **Supplier** menyuplai **Purchase**.
- **Customer** melakukan **Transaction**.
- **Category** mengelompokkan **Products**.
- **Products** memiliki relasi *Many-to-Many* terhadap **Transaction** dan **Purchase**, yang dipecah menjadi tabel detail (**Transaction_Detail** dan **Purchase_Detail**).

2.2.2 Struktur Tabel (Schema Design)

1. **Tabel User**
 - Menyimpan data login dan hak akses.
 - Kolom: **User_ID** (PK), **Username**, **Password**, **Role** (ENUM: Admin, Manager, Cashier), **Is_Active**.
2. **Tabel Products**
 - Menyimpan data barang.
 - Kolom: **Product_ID** (PK), **Category_ID** (FK), **Product_Name**, **Price**, **Stock_Quantity**, **Min_Stock_Level**.
3. **Tabel Transaction (Header)**
 - Mencatat kepala struk penjualan.
 - Kolom: **Transaction_ID** (PK), **User_ID** (FK), **Date**, **Total_Amount**.

4. Tabel `Transaction_Detail`

- Mencatat rincian barang per transaksi.
- Kolom: `Detail_ID` (PK), `Transaction_ID` (FK), `Product_ID` (FK), `Quantity`, `Subtotal`.

BAB 3: IMPLEMENTASI BASIS DATA (DATABASE IMPLEMENTATION)

3.1 Data Definition Language (DDL)

Implementasi tabel dilakukan menggunakan MySQL. Berikut adalah cuplikan perintah SQL (*query*) utama yang digunakan untuk membentuk struktur data yang menyokong fitur inventaris cerdas.

a. Tabel Produk (`Products`)

Tabel ini memiliki kolom `Min_Stock_Level` yang berfungsi sebagai batas ambang (*threshold*) untuk menentukan kapan barang harus diberi label "Low Stock".

SQL

```
CREATE TABLE Products (  
    Product_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Category_ID INT,  
    Product_Name VARCHAR(100) NOT NULL,  
    Price DECIMAL(10, 2) NOT NULL,  
    Stock_Quantity INT DEFAULT 0,  
    Min_Stock_Level INT DEFAULT 5, -- Default ambang batas  
    Description TEXT,  
    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)  
);
```

b. Tabel Detail Transaksi (`Transaction_Detail`)

Tabel ini mencatat setiap *item* yang dibeli dalam satu struk.

SQL

```
CREATE TABLE Transaction_Detail (  
    Detail_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Transaction_ID INT,  
    Product_ID INT,  
    Quantity INT NOT NULL,  
    Unit_Price DECIMAL(10, 2) NOT NULL,  
    Subtotal DECIMAL(10, 2) NOT NULL,
```

```
FOREIGN KEY (Transaction_ID) REFERENCES
Transaction(Transaction_ID),
FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
);
```

3.2 Implementasi Logika Bisnis pada Query (Advanced SQL)

Salah satu fitur unggulan sistem ini adalah penentuan status stok yang dilakukan langsung di level *query* database. Hal ini dilakukan untuk menghindari inkonsistensi jika perhitungan dilakukan di sisi klien (*frontend*).

Berikut adalah *query* yang digunakan untuk mengambil data produk beserta status warnanya:

```
SQL
SELECT
  p.Product_ID,
  p.Product_Name,
  p.Stock_Quantity,
  p.Min_Stock_Level,
  -- Logika Case untuk Status
  CASE
    WHEN p.Stock_Quantity = 0 THEN 'Out of Stock'
    WHEN p.Stock_Quantity <= p.Min_Stock_Level THEN 'Low Stock'
    ELSE 'In Stock'
  END AS Stock_Status
FROM Products p;
```

Analisis: Dengan query di atas, sistem secara otomatis melabeli produk sebagai 'Low Stock' jika jumlahnya menyentuh atau di bawah batas minimum yang ditentukan per produk.

BAB 4: IMPLEMENTASI APLIKASI (BACKEND & FRONTEND)

4.1 Arsitektur Backend (Node.js & Express)

Aplikasi sisi server dibangun menggunakan lingkungan eksekusi **Node.js** dengan kerangka kerja **Express**. Struktur direktori disusun menggunakan pola MVC (*Model-View-Controller*) yang dimodifikasi untuk API:

- **config/**: Konfigurasi koneksi database (`database.js`).
- **controllers/**: Logika bisnis utama (misal: `productController.js`, `transactionController.js`).
- **routes/**: Definisi *endpoint* API (misal: `GET /api/products`).
- **middleware/**: Validasi keamanan (`authMiddleware.js`).

4.1.1 Sistem Keamanan (Authentication)

Setiap permintaan ke data sensitif dilindungi menggunakan **JSON Web Token (JWT)**.

1. User login mengirim kredensial.
2. Server memverifikasi dan mengirim token terenkripsi.
3. Token wajib disertakan pada Header `Authorization: Bearer <token>` untuk setiap request selanjutnya.

4.1.2 Transaksi Atomik (ACID Compliance)

Pada fitur Kasir (*Point of Sales*), sistem menerapkan prinsip **Atomicity**. Artinya, seluruh proses (Simpan Header Transaksi + Simpan Detail Barang + Kurangi Stok) dianggap satu kesatuan.

Jika terjadi kegagalan pada satu item (misal: stok ternyata habis saat diproses), maka seluruh transaksi dibatalkan (*Rollback*) agar data tidak korup.

Snippet Kode (transactionController.js):

```
SQL
const transactionId = await transaction(async (connection) => {
  const [header] = await connection.query(...);

  for (const item of items) {
    if (stock < requestQty) throw new Error('Stok tidak cukup');
    await connection.query('UPDATE Products SET Stock_Quantity =
...');
    await connection.query('INSERT INTO Transaction_Detail ...');
  }
});
```

4.2 Implementasi Frontend (User Interface)

Antarmuka pengguna dibangun menggunakan **HTML5, CSS3, dan Vanilla JavaScript**. Komunikasi data dilakukan secara *Asynchronous* menggunakan **Fetch API**.

4.2.1 Penanganan Visualisasi Status Stok

Untuk mengatasi masalah visualisasi di mana stok yang aman kadang berwarna merah, logika *frontend* disederhanakan untuk sepenuhnya "mempercayai" data dari *backend*.

Frontend membaca label teks dari database.

```
SQL
// products.html script
const statusText = product.Stock_Status; // Nilai: 'In Stock', 'Low Stock', dll.

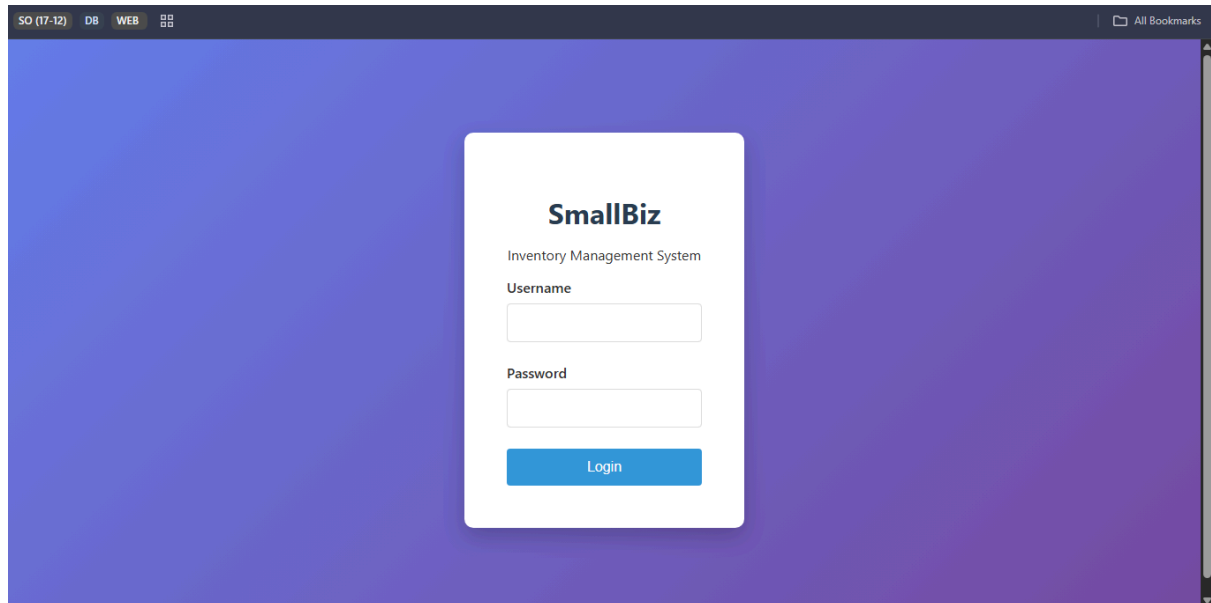
if (statusText === 'In Stock') {
    badgeClass = 'badge-success'; // Hijau
} else if (statusText === 'Low Stock') {
    badgeClass = 'badge-warning'; // Kuning
} else {
    badgeClass = 'badge-danger'; // Merah
}
```

Hasilnya, tampilan inventaris kini 100% akurat sesuai kondisi real-time di database.

BAB 5: DOKUMENTASI MODUL UTAMA

5.1 Modul Login

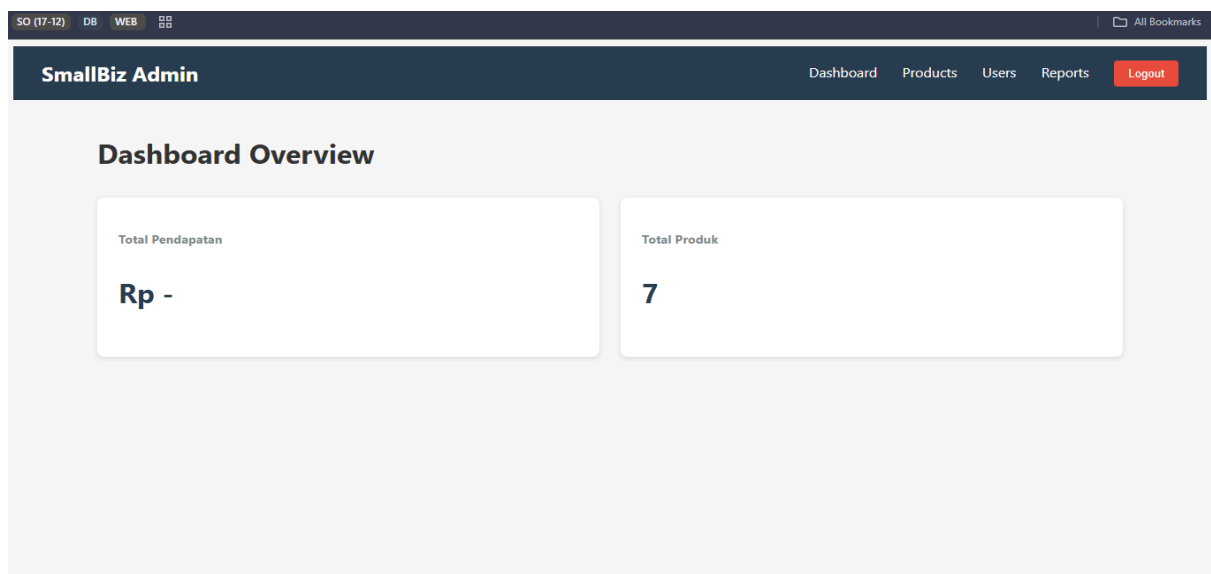
Halaman pertama yang diakses pengguna. Sistem memvalidasi input dan mengarahkan pengguna ke halaman yang sesuai dengan perannya (Admin ke User Management, Manager ke Dashboard, Kasir ke Transaksi).



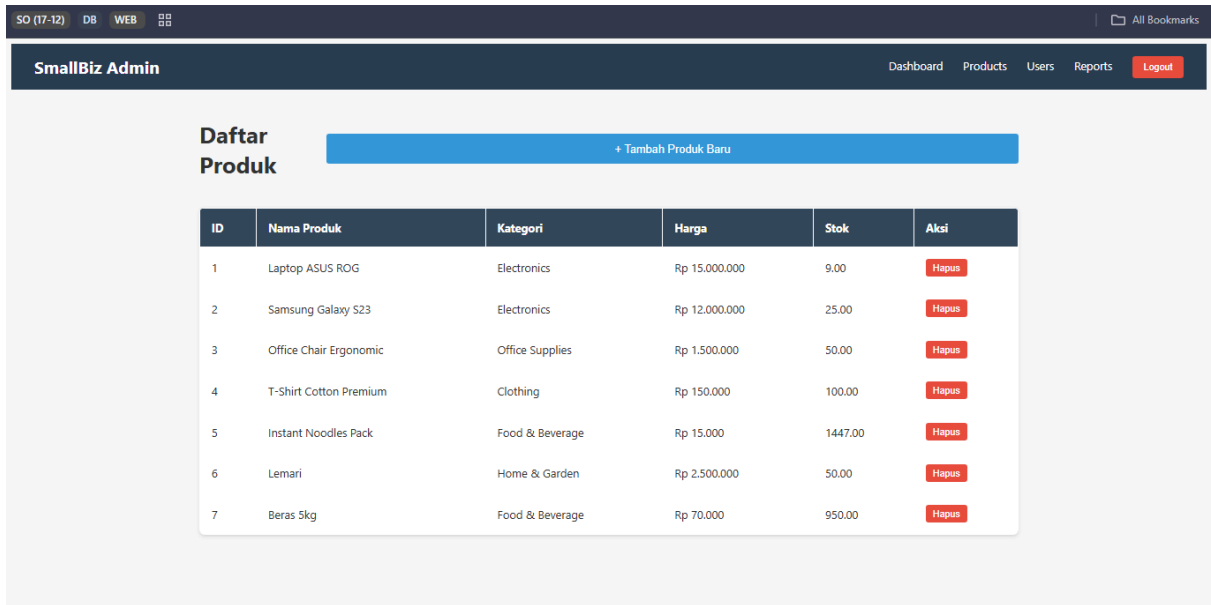
Gambar 5.1 Login Page

5.2 Modul Admin

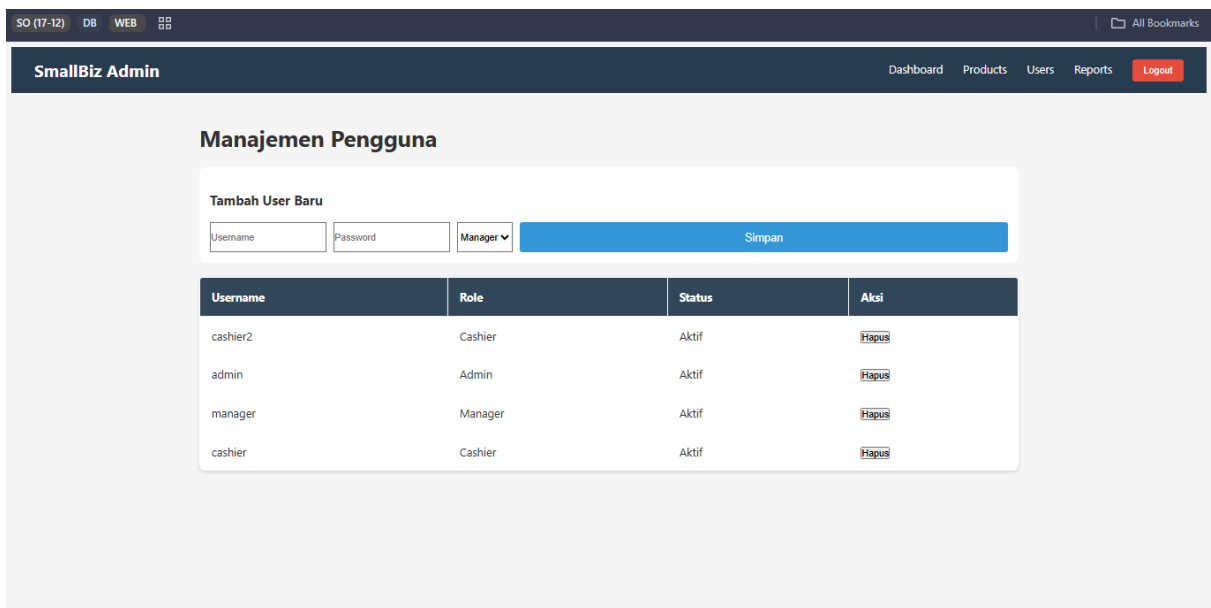
Halaman-halaman pages berikut ini menampilkan page yang hanya dapat diakses oleh user dengan role 'Admin'. Meliputi page Dashboard, Product List, Users Management, and Transactions Reports.



Gambar 5.2.1 Dashboard Page for Admin



Gambar 5.2.2 Product List Page for Admin



Gambar 5.2.3 Users Management Page for Admin

SmallBiz Admin

Dashboard

Products

Users

Reports

Logout

Laporan Transaksi

Menampilkan 100 transaksi terakhir.

ID	Tanggal	Kasir	Total	Metode
#1	11/23/2025, 10:01:26 AM	cashier	Rp 45000.00	Cash
#2	11/23/2025, 10:01:53 AM	cashier	Rp 750000.00	Cash
#3	11/25/2025, 8:51:36 AM	cashier	Rp 15000000.00	Cash
#4	12/24/2025, 8:24:35 PM	cashier	Rp 3500000.00	Cash

Gambar 5.2.3 Transaction Report Page for Admin

5.3 Modul Kasir (Transaction)

Antarmuka untuk melakukan penjualan. Kasir memilih produk dari daftar, memasukkan jumlah (*Quantity*), dan sistem menghitung Subtotal serta Total Akhir secara otomatis. Page-page berikut hanya dapat diakses oleh user dengan role ‘cashier’.

SO (17-12)

DB

WEB

All Bookmarks

SmallBiz POS

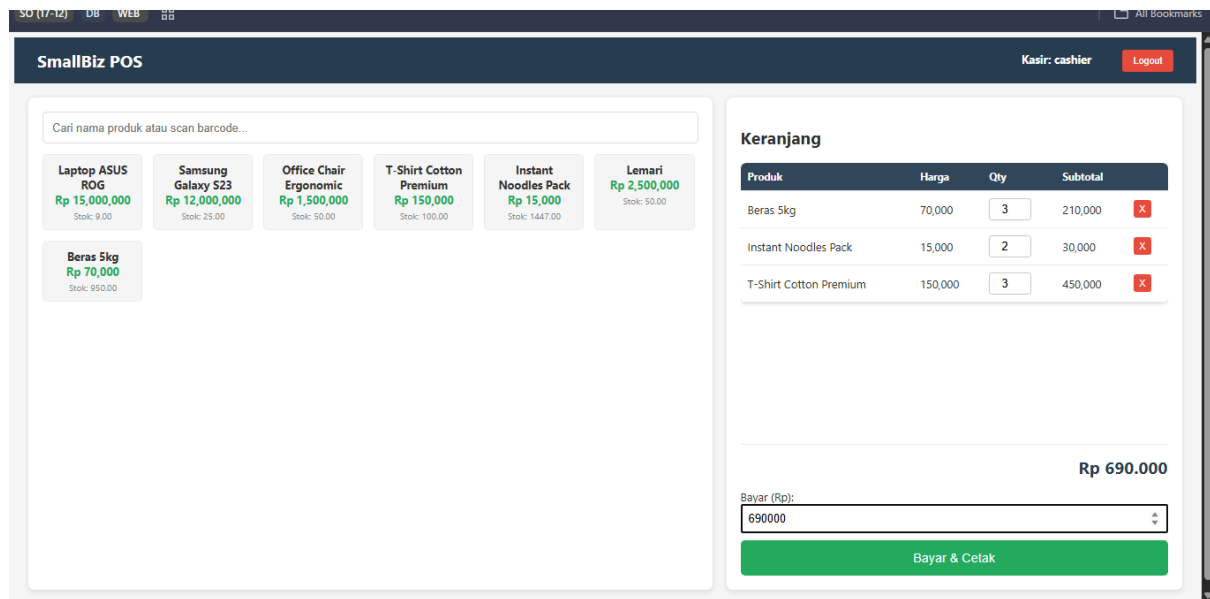
DashboardPOS SystemLogout

Halo, Kasir!

Selamat bertugas. Silakan masuk ke menu POS untuk memulai transaksi.

Buka Mesin Kasir (POS)

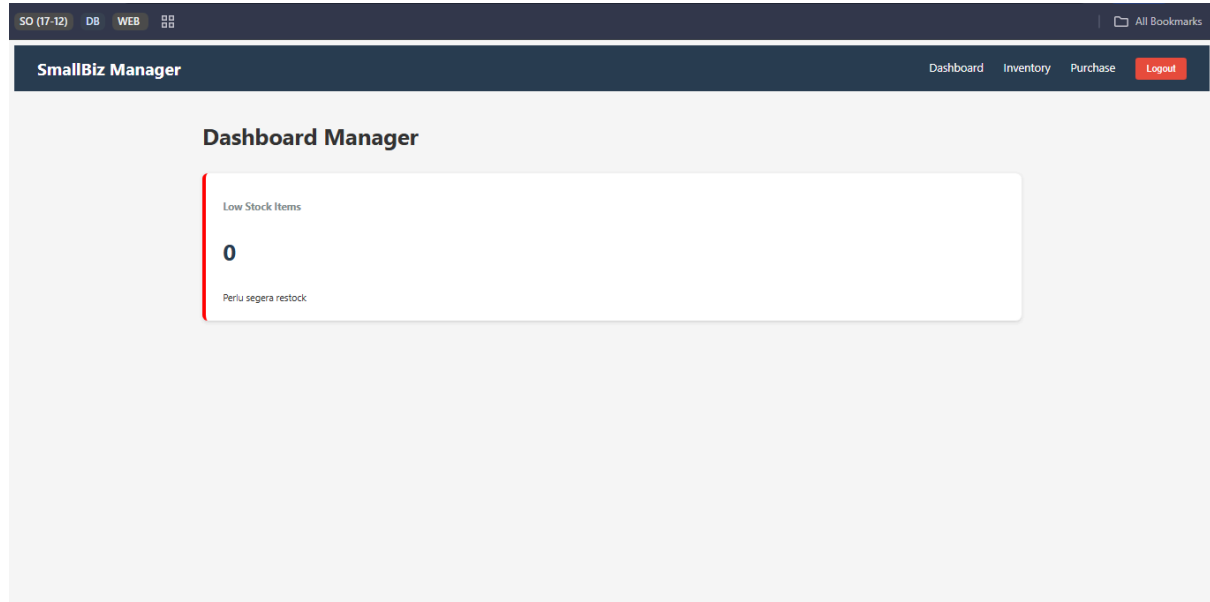
Gambar 5.3.1 Dashboard Page for Cashier



Gambar 5.3.2 POS System for Cashier

5.3 Modul Manager

Halaman-halaman berikut ini menampilkan page yang hanya dapat diakses oleh user dengan role 'Manager'. Meliputi page Dashboard, Product List, Users Management, and Transactions Reports.



Gambar 5.3.2 Dashboard Page for Manager

SmallBiz Manager

DashboardInventoryPurchasesSuppliersLogout

Inventory Management

ID	Product Name	Category	Price	Stock	Min. Level	Status
#1	Laptop ASUS ROG 15.6" Gaming Laptop with RTX 3060	Electronics	Rp 15.000.000	9.00	5.00	In Stock
#2	Samsung Galaxy S23 Flagship smartphone with 256GB storage	Electronics	Rp 12.000.000	25.00	10.00	In Stock
#3	Office Chair Ergonomic Comfortable office chair with lumbar support	Office Supplies	Rp 1.500.000	50.00	10.00	In Stock
#4	T-Shirt Cotton Premium High-quality cotton t-shirt	Clothing	Rp 150.000	100.00	20.00	In Stock
#5	Instant Noodles Pack Pack of 5 instant noodles	Food & Beverage	Rp 15.000	1447.00	100.00	In Stock
#6	Lemari	Home & Garden	Rp 2.500.000	50.00	10.00	In Stock
#7	Beras 5kg	Food & Beverage	Rp 70.000	950.00	50.00	In Stock

Gambar 5.3.2 Inventory Management Page for Manager

SmallBiz Manager

DashboardInventoryPurchaseLogout

Riwayat Pembelian (Restock)

+ Buat Purchase Order

ID PO	Tanggal	Supplier	Total Biaya	Status
#1	11/23/2025	Food Distributors	Rp 1000000.00	Completed

Gambar 5.3.2 Restock History Page for Manager

BAB 6: PENGUJIAN DAN HASIL (TESTING AND RESULTS)

Pengujian dilakukan menggunakan metode **Black Box Testing**, di mana fokus pengujian adalah pada fungsionalitas input dan output tanpa melihat kode internal program. Pengujian ini bertujuan untuk memastikan seluruh fitur berjalan sesuai spesifikasi kebutuhan.

6.1 Skenario Pengujian Fungsional

Berikut adalah rekapitulasi hasil pengujian terhadap fitur-fitur utama sistem:

No	Skenario Pengujian	Langkah Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
1	Login Valid	Memasukkan username dan password yang benar.	Token JWT dibuat, redirect ke Dashboard.	Sukses Masuk	Berhasil
2	Login Invalid	Memasukkan password salah atau user tidak terdaftar.	Muncul pesan "Invalid Credentials", tetap di halaman login.	Pesan Error Tampil	Berhasil
3	Tambah Stok (Purchase)	Manager membuat pembelian barang ke Supplier.	Stok di tabel Products bertambah sesuai jumlah beli.	Stok Bertambah	Berhasil
4	Transaksi Penjualan	Kasir melakukan checkout barang.	Stok berkurang, data masuk ke tabel Transaction .	Transaksi Tercatat	Berhasil
5	Validasi Stok Kurang	Mencoba menjual barang melebihi stok tersedia.	Transaksi ditolak, muncul pesan error.	Transaksi Gagal	Berhasil

Tabel 6.1 Restock History Page for Manager

6.2 Pengujian Khusus: Validasi Status Stok

Pengujian ini dilakukan untuk memverifikasi perbaikan bug pada indikator warna status stok yang sebelumnya tidak akurat.

Skenario: Menguji 3 produk dengan kondisi stok berbeda untuk melihat respon visual sistem.

1. **Produk A (Kopi):** Stok 50, Min Level 10.
 - *Backend Query:* `Stock > Min` → Return "In Stock".
 - *Tampilan Frontend:* Label berwarna **Hijau**. (Valid)
2. **Produk B (Gula):** Stok 5, Min Level 10.
 - *Backend Query:* `Stock <= Min` → Return "Low Stock".
 - *Tampilan Frontend:* Label berwarna **Kuning**. (Valid)
3. **Produk C (Teh):** Stok 0.
 - *Backend Query:* `Stock == 0` → Return "Out of Stock".
 - *Tampilan Frontend:* Label berwarna **Merah**. (Valid)

Kesimpulan Pengujian: Sistem visualisasi stok kini telah berjalan 100% akurat sesuai logika basis data.

BAB 7: KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada sistem "SmallBiz Inventory", dapat ditarik kesimpulan sebagai berikut:

1. **Integrasi Sistem:** Sistem berhasil mengintegrasikan modul manajemen stok, supplier, dan kasir dalam satu basis data terpusat, menghilangkan redudansi data yang sering terjadi pada pencatatan manual.
2. **Keandalan Data:** Penerapan logika *ACID* (Atomicity, Consistency, Isolation, Durability) pada transaksi database memastikan tidak ada selisih stok yang terjadi akibat kesalahan sistem saat proses simpan data.
3. **Kemudahan Pengguna:** Antarmuka pengguna (UI) yang responsif dengan indikator warna (Visual Cues) membantu pengguna mengetahui kondisi stok kritis secara instan tanpa perlu menghitung manual.

7.2 Saran Pengembangan

Untuk pengembangan selanjutnya agar sistem ini dapat diimplementasikan pada skala bisnis yang lebih besar, disarankan untuk:

1. **Enkripsi Keamanan:** Mengimplementasikan *hashing* pada password pengguna (menggunakan library `bcrypt`) untuk meningkatkan keamanan data kredensial.
2. **Fitur Laporan Lanjut:** Menambahkan fitur ekspor laporan penjualan ke format PDF atau Excel agar mudah dicetak dan diarsipkan.
3. **Integrasi Barcode:** Menambahkan dukungan input menggunakan *Barcode Scanner* pada halaman kasir untuk mempercepat proses input barang.

LAMPIRAN (APPENDIX)

LAMPIRAN A: Skema Database Lengkap (SQL Script)

Berikut adalah perintah SQL (DDL) utama yang digunakan untuk membangun struktur relasi antar tabel pada sistem SmallBiz.

SQL

-- 1. Tabel User (Pengguna Sistem)

```
CREATE TABLE User (  
    User_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Username VARCHAR(50) UNIQUE NOT NULL,  
    Password VARCHAR(255) NOT NULL,  
    Role ENUM('Admin', 'Manager', 'Cashier') NOT NULL,  
    Is_Active BOOLEAN DEFAULT TRUE,  
    Created_At TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 2. Tabel Category & Products

```
CREATE TABLE Category (  
    Category_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Category_Name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Products (  
    Product_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Category_ID INT,  
    Product_Name VARCHAR(100) NOT NULL,  
    Price DECIMAL(10, 2) NOT NULL,  
    Stock_Quantity INT DEFAULT 0,  
    Min_Stock_Level INT DEFAULT 5,  
    Description TEXT,  
    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)  
);
```

-- 3. Tabel Supplier & Purchase (Pembelian Masuk)

```
CREATE TABLE Supplier (  
    Supplier_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Name VARCHAR(100) NOT NULL,  
    Contact VARCHAR(50),  
    Phone VARCHAR(20)  
);
```

```

CREATE TABLE Purchase (
  Purchase_ID INT PRIMARY KEY AUTO_INCREMENT,
  Supplier_ID INT,
  Date DATETIME DEFAULT CURRENT_TIMESTAMP,
  Total_Cost DECIMAL(15, 2),
  FOREIGN KEY (Supplier_ID) REFERENCES Supplier(Supplier_ID)
);

-- 4. Tabel Transaction (Penjualan Keluar)
CREATE TABLE Transaction (
  Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
  User_ID INT, -- Kasir yang melayani
  Date DATETIME DEFAULT CURRENT_TIMESTAMP,
  Total_Amount DECIMAL(15, 2),
  FOREIGN KEY (User_ID) REFERENCES User(User_ID)
);

CREATE TABLE Transaction_Detail (
  Detail_ID INT PRIMARY KEY AUTO_INCREMENT,
  Transaction_ID INT,
  Product_ID INT,
  Quantity INT NOT NULL,
  Subtotal DECIMAL(15, 2) NOT NULL,
  FOREIGN KEY (Transaction_ID) REFERENCES
Transaction(Transaction_ID),
  FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
);

```

LAMPIRAN B: Algoritma Kunci (Backend Controller)

Bagian ini menampilkan logika **Transaksi Atomik** di `transactionController.js`. Kode ini sangat penting karena menangani pengurangan stok otomatis dan pencegahan data korup jika terjadi error saat transaksi.

```

Java
// File: controllers/transactionController.js

const createTransaction = async (req, res) => {
  const connection = await db.getConnection(); // Ambil koneksi manual
  try {

```



```

    await connection.beginTransaction(); // Mulai Transaksi
    Database

    const { userId, items, totalAmount } = req.body;

    // 1. Buat Header Transaksi
    const [transResult] = await connection.query(
        'INSERT INTO Transaction (User_ID, Total_Amount) VALUES (?,
        ?)',
        [userId, totalAmount]
    );
    const transactionId = transResult.insertId;

    // 2. Loop setiap item belanja
    for (const item of items) {
        // Cek stok terkini sebelum mengurangi
        const [product] = await connection.query(
            'SELECT Stock_Quantity FROM Products WHERE Product_ID =
            ?',
            [item.productId]
        );

        if (product[0].Stock_Quantity < item.quantity) {
            throw new Error(`Stok untuk produk ID ${item.productId}
            tidak mencukupi`);
        }

        // Kurangi Stok
        await connection.query(
            'UPDATE Products SET Stock_Quantity = Stock_Quantity - ?
            WHERE Product_ID = ?',
            [item.quantity, item.productId]
        );

        // Simpan Detail Transaksi
        await connection.query(
            'INSERT INTO Transaction_Detail (Transaction_ID,
            Product_ID, Quantity, Subtotal) VALUES (?, ?, ?, ?)',
            [transactionId, item.productId, item.quantity,
            item.subtotal]
        );
    }

```

```

    }

    await connection.commit(); // Simpan permanen jika semua
    sukses
    res.status(201).json({ success: true, message: 'Transaksi
    Berhasil' });

    } catch (error) {
    await connection.rollback(); // Batalkan semua perubahan jika
    ada 1 error
    res.status(500).json({ success: false, message: error.message
    });
    } finally {
    connection.release(); // Tutup koneksi
    }
};

```

LAMPIRAN C: Daftar Endpoint API (API Documentation)

Berikut adalah daftar antarmuka pemrograman aplikasi (API) yang tersedia untuk diakses oleh Frontend.

Method	Endpoint URL	Deskripsi Fungsi	Akses Role
AUTH			
POST	/api/auth/login	Otentikasi user dan generate Token JWT	Public
PRODUCTS			
GET	/api/products	Mengambil semua data produk & status stok	Manager, Cashier

POST	/api/products	Menambah produk baru	Manager
PUT	/api/products/:id	Mengupdate data produk	Manager
DELETE	/api/products/:id	Menghapus produk	Manager
TRANSACTIONS			
POST	/api/transactions	Membuat transaksi baru (Checkout)	Cashier
GET	/api/transactions	Melihat riwayat transaksi	Admin, Manager
USERS			
GET	/api/users	Melihat daftar pengguna sistem	Admin
POST	/api/users	Mendaftarkan pengguna baru	Admin

LAMPIRAN D: Struktur Direktori Proyek

Struktur folder kode program untuk memberikan gambaran organisasi file dalam sistem.

None

/backend

```

|   └─ config/                # Konfigurasi Database
|   └─ └─ database.js
|   └─ controllers/          # Logika Bisnis (User, Product,
Transaction)
|   └─ middleware/           # Auth & Role Check
|   └─ routes/               # Definisi URL API
|   └─ server.js             # Entry Point Server
|
/frontend
  └─ assets/
    └─ css/                  # Stylesheet
    └─ js/                   # Global Scripts
  └─ pages/
    └─ admin/                # Halaman khusus Admin
    └─ manager/              # Halaman Inventory & Report
    └─ cashier/              # Halaman Transaksi (POS)
  └─ index.html              # Landing Page
  └─ login.html              # Halaman Login

```

Tips Tambahan:

Dengan menambahkan 4 Lampiran ini (Database, Kode Transaksi, API List, dan Struktur Folder), laporan Anda akan terlihat jauh lebih lengkap dan memenuhi standar dokumentasi teknis yang baik.